# 大数据处理架构Hadoop

丁玥
上海交通大学计算机学院
dingyue@sjtu.edu.cn

2026年秋

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# 目 录




1 Hadoop简介

2 分布式文件系统HDFS

3 资源管理平台YARN

4 分布式计算框架MapReduce

5 搭建Hadoop

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# 目 录




1 Hadoop简介

2 分布式文件系统HDFS

3 资源管理平台YARN

4 分布式计算框架MapReduce

5 搭建Hadoop

## Hadoop的前世今生

- 一位名叫**Doug Cutting**的美国工程师迷上了搜索引擎，他做了一个用于文本搜索的函数库，命名为**Lucene**




- 早期的时候，这个项目被发布在**Doug Cutting**的个人网站和**SourceForge**（一个开源软件网站）。2001年底，**Lucene**成为Apache软件基金会**jakarta**项目的一个子项目



Lucene

The Apache Software Foundation
http://www.apache.org/

## Hadoop的前世今生

- 2004年，**Doug Cutting**再接再励，在**Lucene**的基础上，和Apache开源伙伴Mike Cafarella合作，开发了一款可以代替当时的主流搜索的开源搜索引擎，命名为**Nutch**




- 随着时间的推移，无论是**Google**还是**Nutch**，都面临搜索对象“体积”不断增大的问题。尤其是**Google**，作为互联网搜索引擎，需要存储大量的网页，并不断优化自己的搜索算法，提升搜索效率。
  - **在这个过程中，Google确实找到了不少好办法，并且无私地分享了出来**

## Hadoop的前世今生

- 2003年，Google发表了一篇技术学术论文，公开介绍了自己的谷歌文件系统GFS（Google File System）
- 第二年，也就是2004年，Doug Cutting基于Google的GFS论文，实现了**分布式文件存储系统**，并将它命名为NDFS（Nutch Distributed File System）



## Hadoop的前世今生

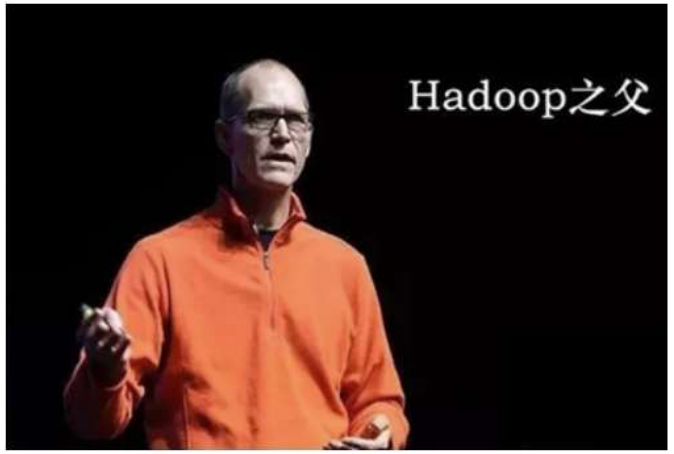
- 还是**2004**年，**Google**又发表了一篇技术学术论文，介绍自己的**MapReduce**编程模型，主要用于大规模数据集（大于**1TB**）的并行分析运算
- 第二年（**2005**年），**Doug Cutting**又基于**MapReduce**，在**Nutch**搜索引擎实现了上述功能




- 2006年，Yahoo为了研发自己的搜索引擎，放弃使用谷歌并招安了**Doug Cutting**

## Hadoop的诞生

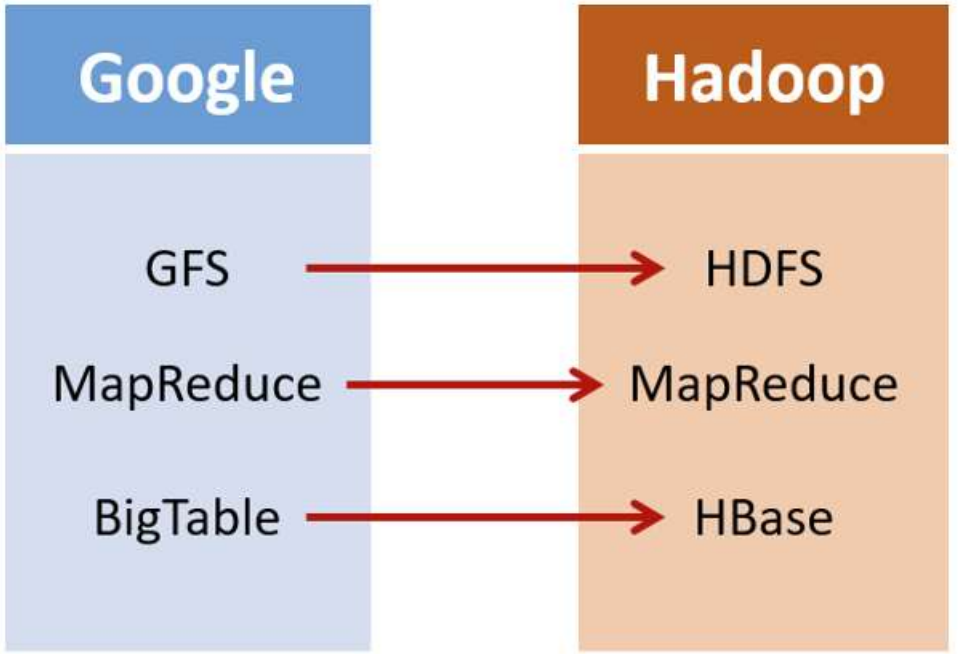
- 加盟Yahoo之后，Doug Cutting将NDFS和MapReduce进行了升级改造，并重新命名为Hadoop（NDFS也改名为HDFS，全称是Hadoop Distributed File System）




- Hadoop这个名字，实际上是Doug Cutting他儿子的黄色玩具大象的名字。所以，Hadoop的Logo，就是一只奔跑的黄色大象

## Hadoop的诞生

- 还是**2006**年，Google又发论文介绍了自己的**BigTable**，这是一种分布式数据存储系统，用来处理海量数据的非关系型数据库。
- **Doug Cutting**当然没有放过，在自己的**Hadoop**系统里面，引入了**BigTable**，并命名为**Hbase**
- Hadoop的核心部分，基本上都有Google的影子



## Hadoop的诞生

- 2008年1月，Hadoop正式成为Apache基金会的顶级项目
- 同年2月，Yahoo宣布建成了一个拥有1万个内核的Hadoop集群，并将自己的搜索引擎产品部署在上面
- 7月，Hadoop打破世界纪录，成为最快排序1TB数据的系统，用时209秒。此后，Hadoop便进入了高速发展期

# Hadoop简介

- Hadoop现在是Apache软件基金会旗下的一个开源分布式计算平台，为用户提供了系统底层细节透明的分布式基础架，Hadoop基于Java语言开发，具有很好的跨平台特性
- Hadoop的核心是分布式文件系统HDFS和MapReduce
- Hadoop被公认为行业大数据标准开源软件，在分布式环境下提供了海量数据的处理能力
- 几乎所有主流厂商都围绕Hadoop提供开发工具、开源软件、商业化工具和技术服务，如谷歌、雅虎、微软、思科、淘宝等，都支持Hadoop

## Hadoop的特性

- **高可靠和高容错性：**采用冗余数据存储方式，即使一个副本发生故障，其他副本也可以保证正常对外提供服务
- **高效性：**能够高效地处理PB级数据。Hadoop能够在节点之间动态地移动数据，并保证各个节点的动态平衡
- **高可扩展性：**可以扩展到数以千计的计算机节点

## Hadoop的国内外应用

- **Yahoo**是Hadoop的最大支持者，截至2012年，Yahoo的Hadoop机器总节点数目超过**42000**个，有超过**10万**的核心CPU在运行Hadoop。
- **Facebook**使用Hadoop存储内部日志与多维数据，并以此作为报告、分析和机器学习的数据源。目前Hadoop集群的机器节点超过**1400**台，共计**11200**个核心CPU，超过**15PB**原始存储容量，每个商用机器节点配置了**8**核CPU，**12TB**数据存储
- **百度**在2012年总的集群规模达到近十个，单集群超过**2800**台机器节点，Hadoop机器总数有上万台机器，总的存储容量超过**100PB**，已经使用的超过**74PB**，每天提交的作业数目有数千个之多，每天的输入数据量已经超过**7500TB**，输出超过**1700TB**
- **阿里巴巴**的Hadoop集群截至2012年大约有**3200**台服务器，大约**30000**个物理CPU核心，总内存**100TB**，总的存储容量超过**60PB**，每天的作业数目超过**150000**个
- 截至2025年底，Uber每日约 **670万个YARN容器**、**40万个Spark应用**

# Hadoop的版本

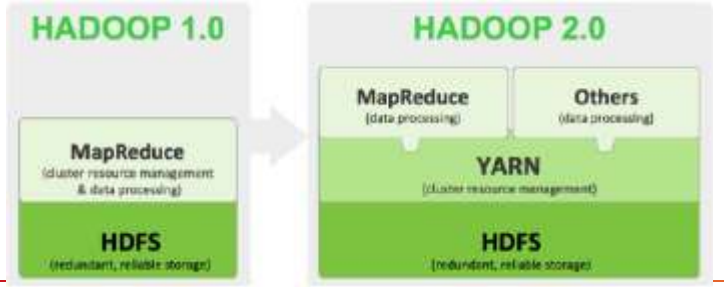
- **Hadoop发行版本**分为**开源社区版**和**商业版**。
- **社区版**是指由Apache**软件基金会维护的版本**，是官方维护的版本体系。
- **商业版**Hadoop是指由**第三方商业公司**在社区版Hadoop基础上进行了一些修改、整合以及各个服务组件兼容性测试而发行的版本。

# Hadoop的版本

- **Hadoop**自诞生以来，主要分为**Hadoop1、Hadoop2、Hadoop3**三个系列的多个版本，
- 目前市场上主要使用**Hadoop2和3两个版本**
  - Hadoop3的技术更新，性能更优
  - 不少生产集群正从Hadoop2迁移到Hadoop3
  - 升级成本问题，仍然有相当一部分企业保留使用Hadoop2

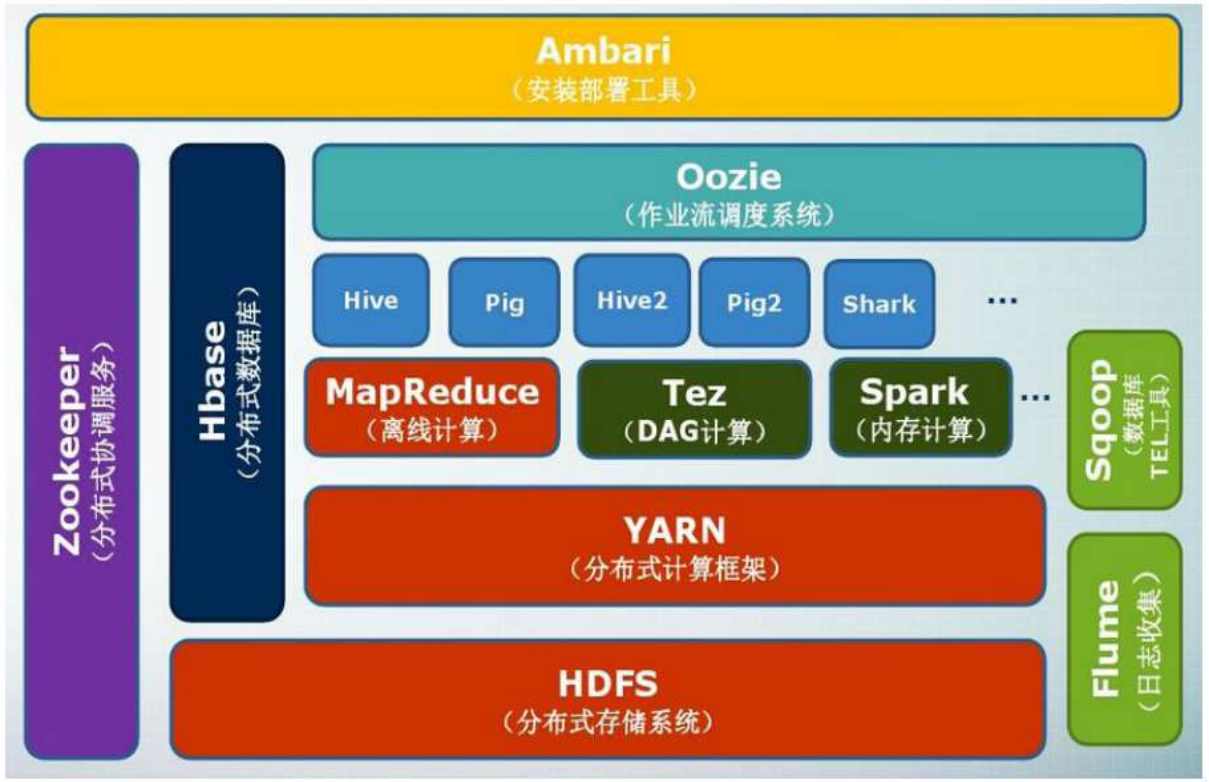
# Hadoop的版本

- **Hadoop1.0内核主要由分布式存储系统HDFS和分布式计算框架MapReduce两个系统组成，而Hadoop2.x版本主要新增了资源管理框架YARN以及其他工作机制的改变**
  - **3.x和2.x具有相同的组成结构，但添加了更多的高级特征以及具有更强的兼容性**
    - **java的最低支持版本**: Hadoop 2.x 为java 7, Hadoop 3.x 为java 8
    - **容错**： Hadoop 2.x可以通过复制（浪费空间）来处理容错；Hadoop 3.x可以通过Erasure编码处理容错
    - **存储Scheme**： Hadoop 2.x 使用3X副本Scheme；Hadoop 3.x支持HDFS中的擦除编码。
    - **YARN时间线服务**： Hadoop 2.x使用具有可伸缩性问题的旧时间轴服务； Hadoop 3.x改进时间线服务v2并提高时间线服务的可扩展性和可靠性
    - ……



# Hadoop的版本

## Hadoop的生态系统




| 组件 | 功能 |
| --- | --- |
| HDFS | 分布式文件系统 |
| MapReduce | 分布式并行编程模型 |
| YARN | 资源管理和调度器 |
| Tez | 运行在YARN之上的下一代Hadoop查询处理框架 |
| Hive | Hadoop上的数据仓库 |
| HBase | Hadoop上的非关系型的分布式数据库 |
| Pig | 一个基于Hadoop的大规模数据分析平台，提供类似SQL的查询语言Pig Latin |
| Sqoop | 用于在Hadoop与传统数据库之间进行数据传递 |
| Oozie | Hadoop上的工作流管理系统 |
| Zookeeper | 提供分布式协调一致性服务 |
| Storm | 流计算框架 |
| Flume | 一个高可用的，高可靠的，分布式的海量日志采集、聚合和传输的系统 |
| Ambari | Hadoop快速部署工具，支持Apache Hadoop集群的供应、管理和监控 |
| Kafka | 一种高吞吐量的分布式发布订阅消息系统，可以处理消费者规模的网站中的所有动作流数据 |
| Spark | 类似于Hadoop MapReduce的通用并行框架 |

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# 目 录




1 Hadoop简介

2 分布式文件系统HDFS

3 资源管理平台YARN

4 分布式计算框架MapReduce

5 搭建Hadoop

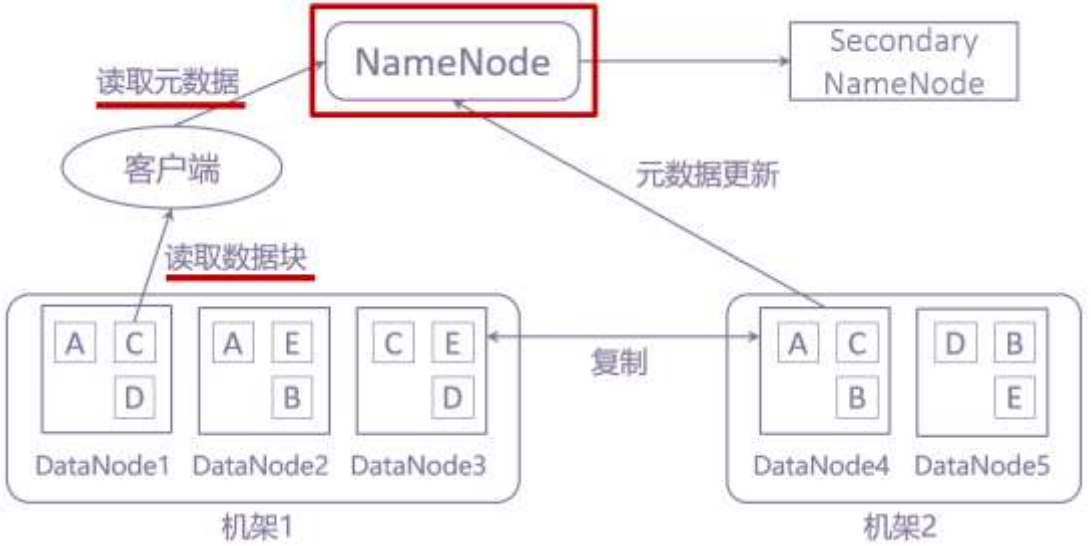
# 分布式文件系统HDFS

- 大数据时代必须解决**海量数据的高效存储问题**
- **分布式文件系统是管理网络中跨多台计算机存储的文件系统**
  - 该系统架构于网络之上，因此比普通磁盘文件系统更为复杂
    - 容忍节点故障
    - 不丢失数据
    - 单一节点的更新与整个文件系统的同步问题

# 分布式文件系统HDFS

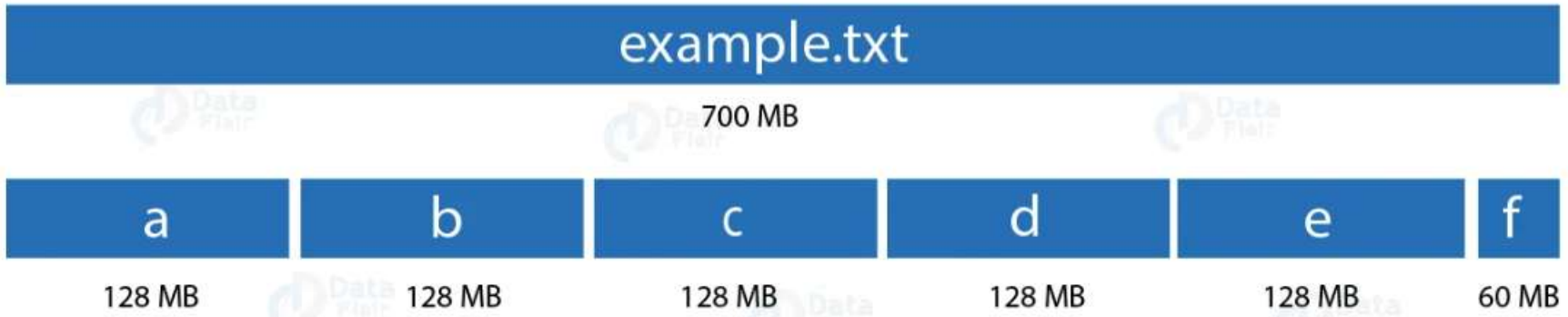
- Hadoop 具有**主从（Master/Slave）拓扑，HDFS作为Hadoop 的分布式文件系统**为Hadoop提供数据存储。HDFS将数据分割称为**块（block）**并以分布式方式存储它们
  - **NameNode 是管理节点，存放文件元数据**
    - 元数据：文件与数据块的映射表（一个文件分成几份）；数据块与数据节点的映射表（数据文件放在哪里）
  - **DataNode是HDFS的工作节点，负责存放数据块**

√在内部，一个文件被**分割成多个数据块并存储在一组从属机器上**
√ **Namenode** 管理对文件或目录的操作例如打开、关闭和重命名
√ **NameNode** 还跟踪块到 **DataNode** 的映射。数据节点服务于来自文件系统客户端的读/写请求
√ **DataNode** 根据 **NameNode** 的需要来创建、删除和复制块。



## HDFS中的块

- **块**是分配给文件的**最小连续存储空间**
- 在 Hadoop 中，**默认块的大小为 128MB （2.7.3版本之后）**




## 我们必须明智地选择 HDFS 块大小

## HDFS中的块大小设置原则：最小化寻址开销，减少网络传输

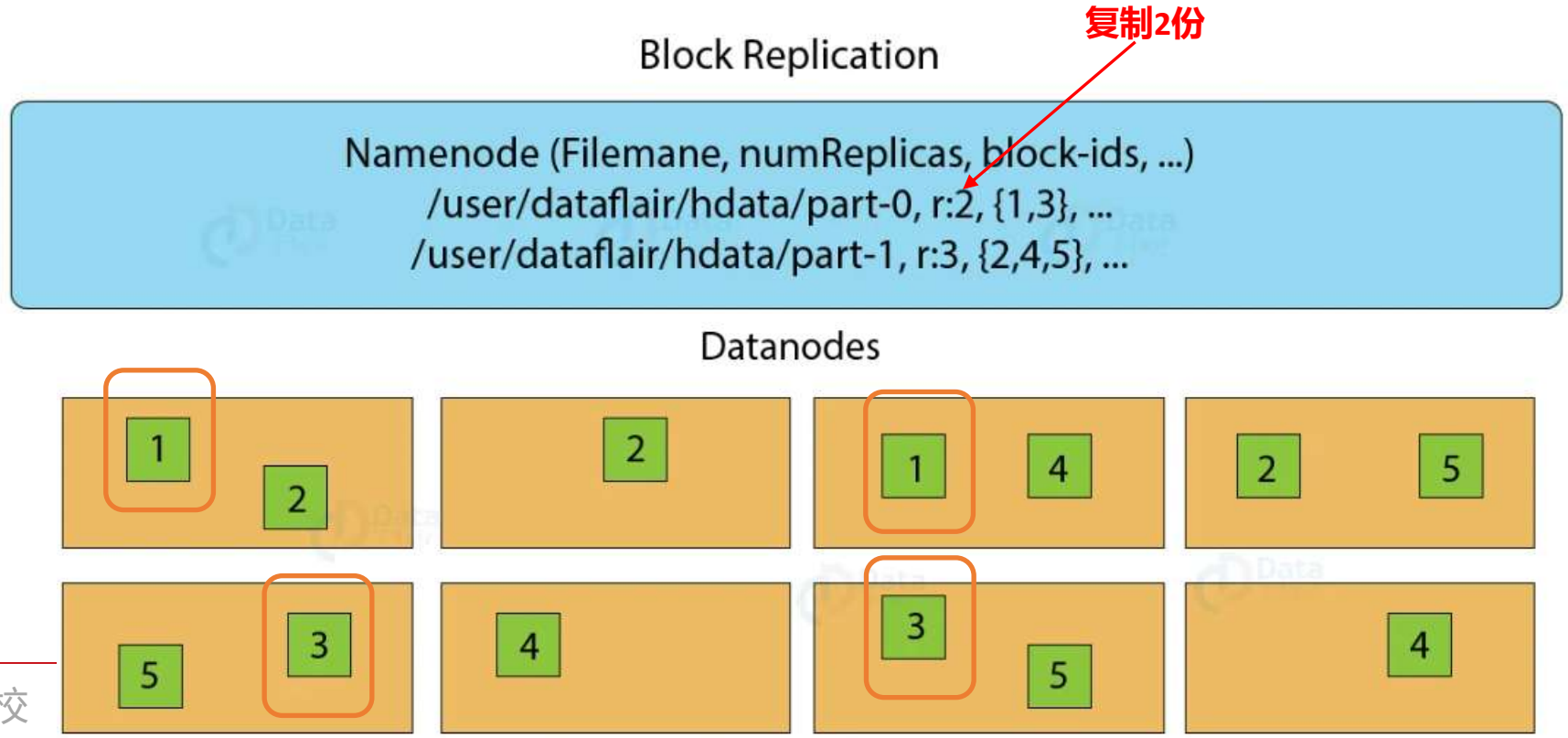
- **如果块的设置过小**：NameNode需要在内存文件中记录DataNode中数据块信息，如果块太小，那么需要维护的数据块信息会更多，而HDFS只有一个NameNode节点，物理内存是有限的；文件块过小，寻址时间增大，导致程序一直在找数据块的开始位置
- **如果块的设置过大**：从磁盘传输数据的时间会明显大于寻址时间，导致程序在处理这块数据时，变得非常慢；MapReduce中的Map任务通常一次只处理一个块中的数据，如果块过大运行速度也会很慢；网络传输时间增长，程序卡顿/超时/无响应；NameNode监管容易判断数据节点死亡，导致集群频繁产生/移除副本，占用CPU，网络，内存资源

## HDFS中块（block）的大小为什么设置为128M?

- 1. HDFS中平均寻址时间大概为10ms；经过测试发现，寻址时间为传输时间的1%时，为最佳状态；所以最佳传输时间为10ms/0.01=1000ms=1s
- 2. 目前磁盘的传输速率普遍为100MB/s ，网卡普遍为千兆网卡传输速率普遍也是100MB/s；计算出最佳block大小：100MB/s x 1s = 100MB，所以我们设定block大小为128MB。
- 4. 实际在工业生产中，需要经过集群之间的具体情况进行设置。比如: 跨物理机/机架之间文件传输速率为200MB/s时，一般设定block大小为256MB，文件传输速率为400MB/s时，一般设定block大小为512MB . **不过最大一般不会超过512MB**，因为目前固态硬盘的读写速率应该不会超过512MB(如果做RAID另行考虑)

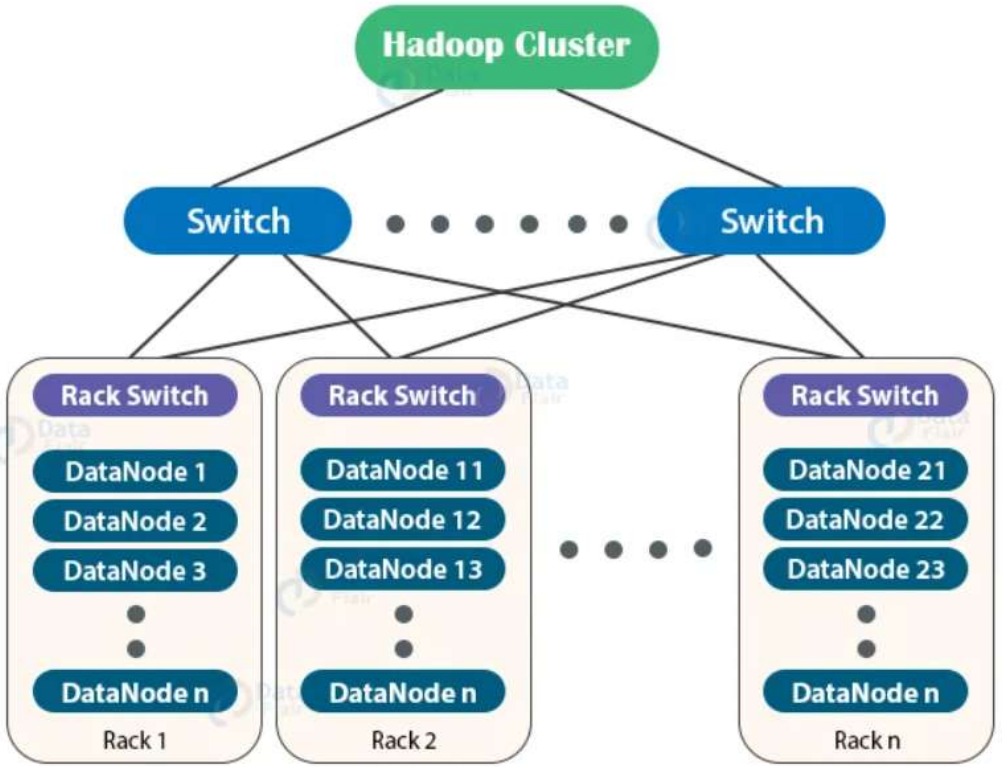
## 复制管理

- HDFS使用复制技术实现**容错能力**
- 复制因子决定存储块的数量。**默认为 3**，但可以配置为任何值
- NameNode 从每个 DataNode 收集块报告



## 数据存放策略（机架感知）

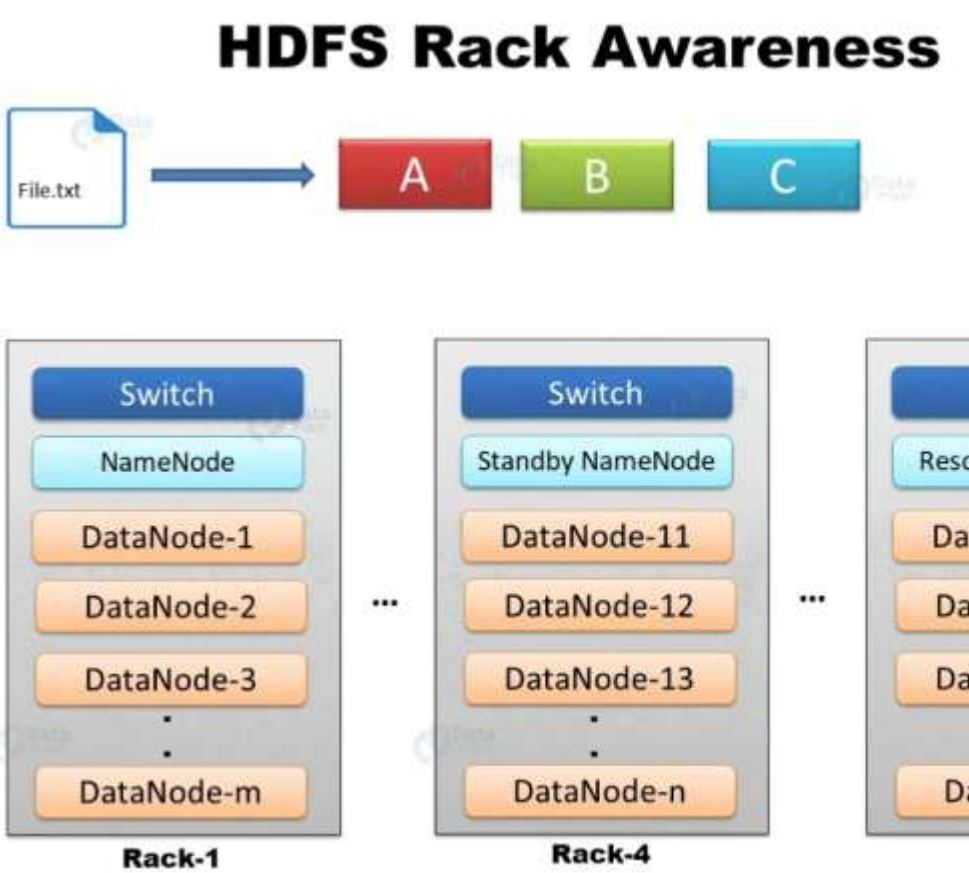
- 一个HDFS集群通常包含多个机架，**不同机架之间**的数据通信**需要**经过交换机或路由器，**同一机架**的不同机器之间数据通信**不需要**交换机或路由器
- 与驻留在不同机架上的 DataNode 之间的通信相比，同一机架上的 DataNode 之间的通信更加高效

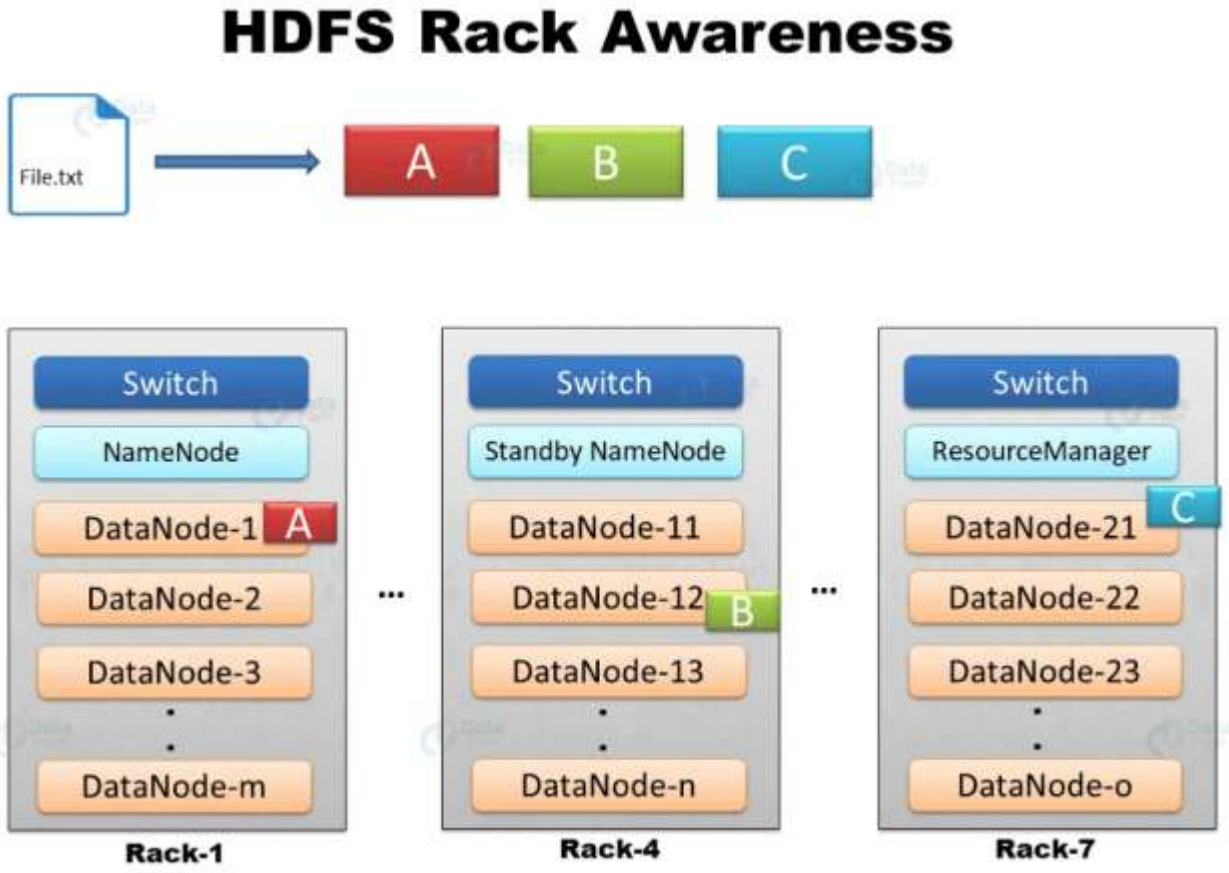


## 数据存放策略（机架感知）

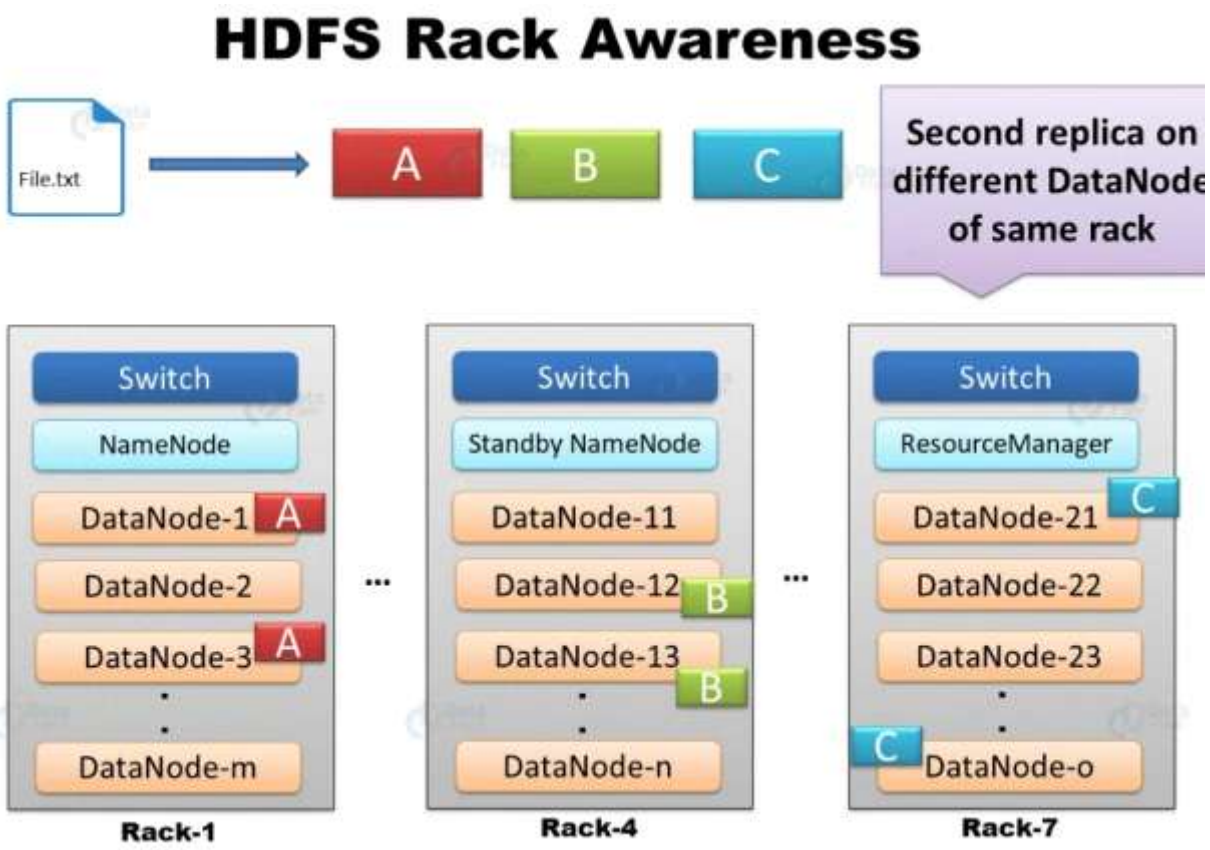
- NameNode 在将副本放置在 HDFS 中时使用**机架感知算法**
- 一个节点上最多只能放置一个副本
- 同一机架上放置的副本不得超过两个
- 对于复制因子为3的常见情况，块复制策略将**第一个副本放在本地机架上**，将第二个副本放在**同一机架上的不同 DataNode上**，将**第三个副本放在不同机架上**

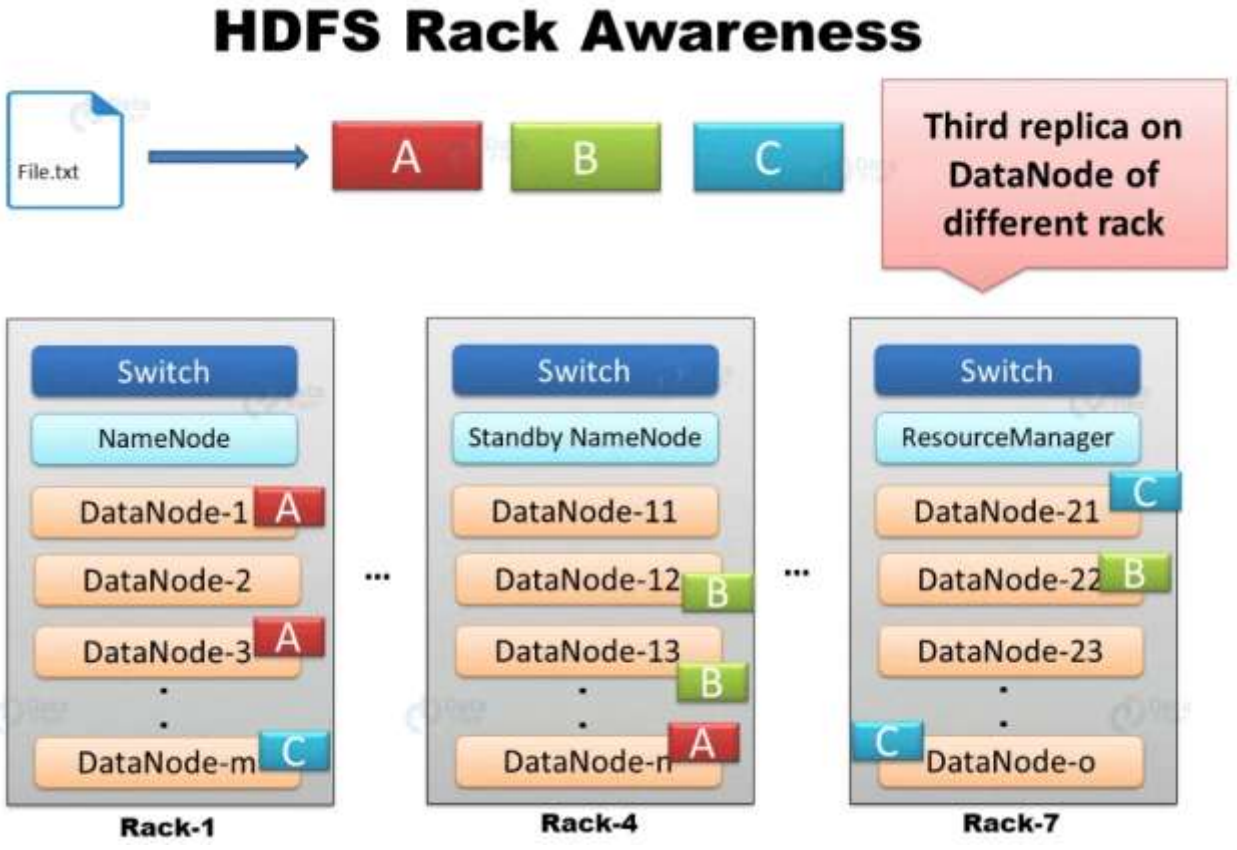
## 数据存放策略（机架感知）


HDFS Rack Awareness
File.txt
A
B
C
Switch
NameNode
DataNode-1
DataNode-2
DataNode-3
DataNode-m
Rack-1
Switch
Standby NameNode
DataNode-11
DataNode-12
DataNode-13
DataNode-n
Rack-4
Switch
ResourceManager
DataNode-21
DataNode-22
DataNode-23
DataNode-o
Rack-7


HDFS Rack Awareness
File.txt
A
B
C
Switch
NameNode
DataNode-1
A
DataNode-2
DataNode-3
DataNode-m
Rack-1
Switch
Standby NameNode
DataNode-11
DataNode-12
B
DataNode-13
DataNode-n
Rack-4
Switch
ResourceManager
DataNode-21
C
DataNode-22
DataNode-23
DataNode-o
Rack-7

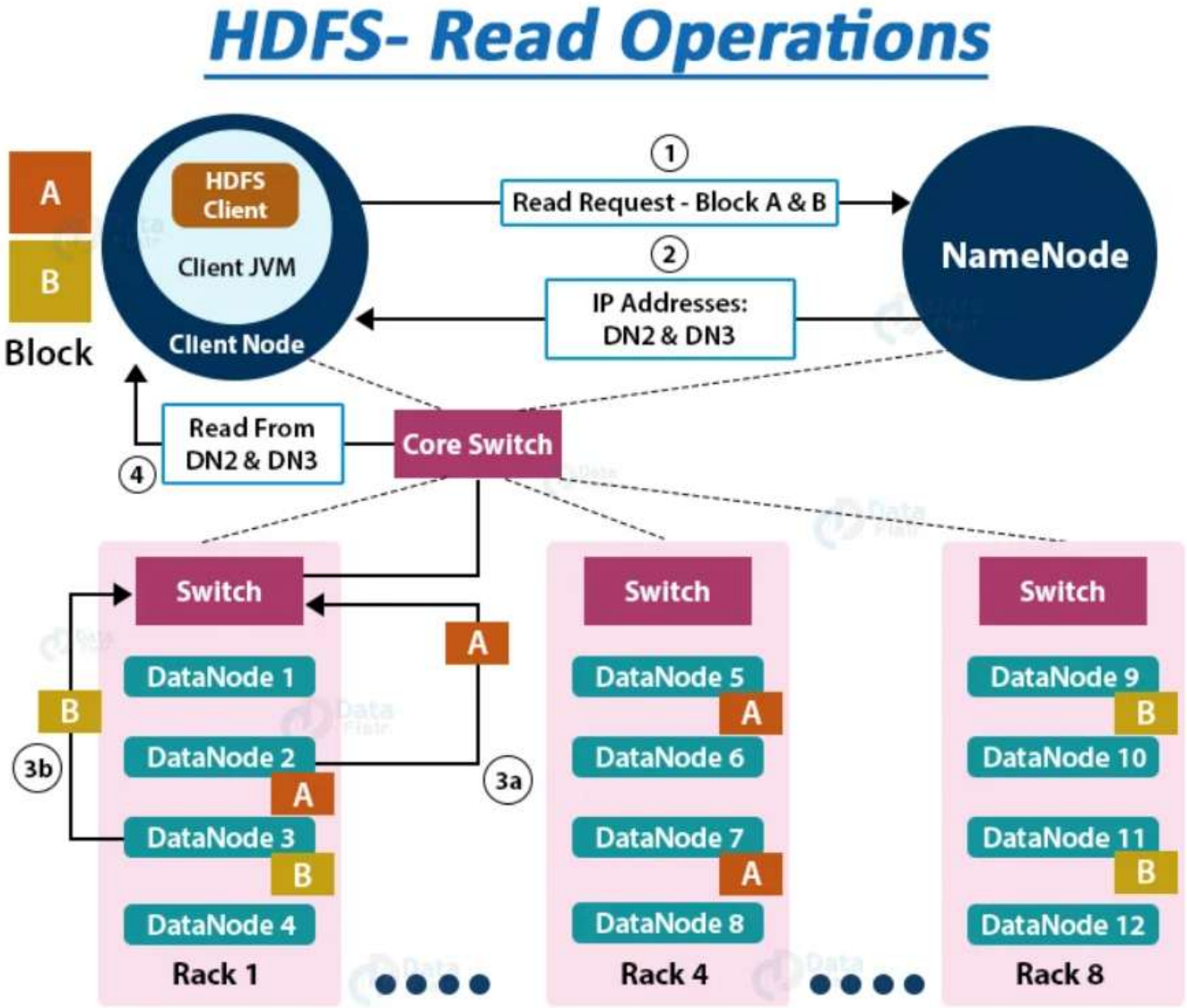
## 数据存放策略（机架感知）


HDFS Rack Awareness
File.txt
A
B
C
Second replica on different DataNode of same rack
Switch
NameNode
DataNode-1
DataNode-2
DataNode-3
DataNode-m
Switch
Standby NameNode
DataNode-11
DataNode-12
DataNode-13
DataNode-n
Switch
ResourceManager
DataNode-21
DataNode-22
DataNode-23
DataNode-o
Rack-1
Rack-4
Rack-7

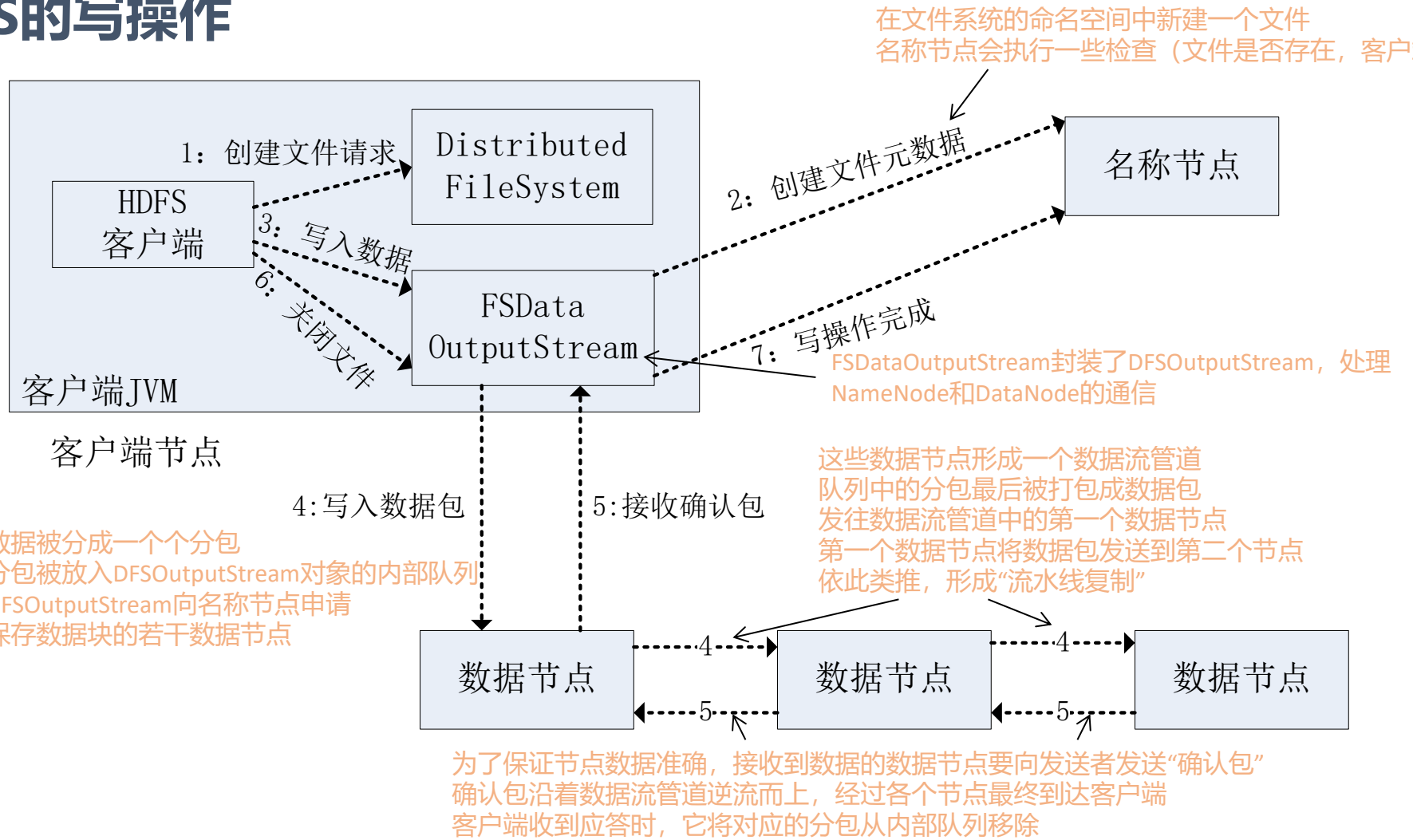

HDFS Rack Awareness
File.txt
A
B
C
Third replica on DataNode of different rack
Switch
NameNode
DataNode-1
DataNode-2
DataNode-3
DataNode-m
Switch
Standby NameNode
DataNode-11
DataNode-12
DataNode-13
DataNode-n
Switch
ResourceManager
DataNode-21
DataNode-22
DataNode-23
DataNode-o
Rack-1
Rack-4
Rack-7

## HDFS的读操作

- 客户端与NameNode交互，询问包含数据块的DataNode的位置
- NameNode发送包含数据块的DataNode的位置，以及提供一个安全令牌
- 客户端向DataNode发送请求，通过显示安全令牌，开始从DataNode读取数据
- 读取完以后关闭



## HDFS的写操作


在文件系统的命名空间中新建一个文件
名称节点会执行一些检查（文件是否存在，客户端权限）
1：创建文件请求
Distributed FileSystem
HDFS 客户端
2：创建文件元数据
名称节点
3：写入数据
6：关闭文件
FSData OutputStream
7：写操作完成
FSDataOutputStream封装了DFSOutputStream，处理NameNode和DataNode的通信
客户端JVM
客户端节点
4:写入数据包
5:接收确认包
这些数据节点形成一个数据流管道
队列中的分包最后被打包成数据包
发往数据流管道中的第一个数据节点
第一个数据节点将数据包发送到第二个节点
依此类推，形成“流水线复制”
数据被分成一个个分包
分包被放入DFSOutputStream对象的内部队列
DFSOutputStream向名称节点申请
保存数据块的若干数据节点
数据节点
4
数据节点
4
数据节点
5
5
为了保证节点数据准确，接收到数据的数据节点要向发送者发送“确认包”
确认包沿着数据流管道逆流而上，经过各个节点最终到达客户端
客户端收到应答时，它将对应的分包从内部队列移除

# HDFS 的容错：故障类型与检测方法

## ① 节点故障




服务器停机或硬件故障

## ② 通信故障



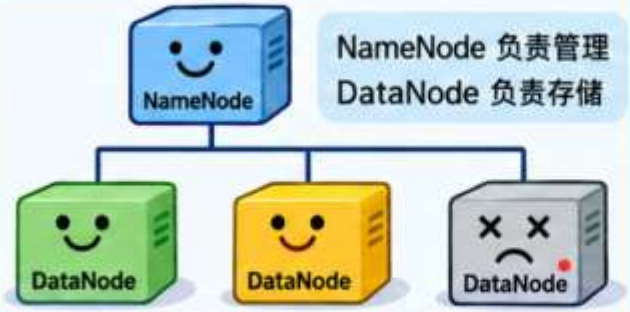

网络中断，无法收发数据

## ③ 数据损坏




传输过程中损坏　存储过程中损坏

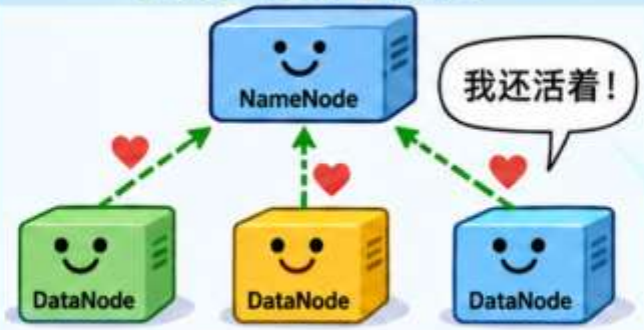
## 重点检测 DataNode




NameNode 负责管理
DataNode 负责存储

单 NameNode 故障会影响服务；
高可用部署可缓解。

## 心跳：报告存活




DataNode 定期发送心跳
例如：每 3 秒一次。

## 超时：判定失联




超时阈值由配置决定。

# HDFS 的容错：读写故障的处理

## 写入：数据包流水线




数据包沿复制管道传输

## 确认：逐级返回应答



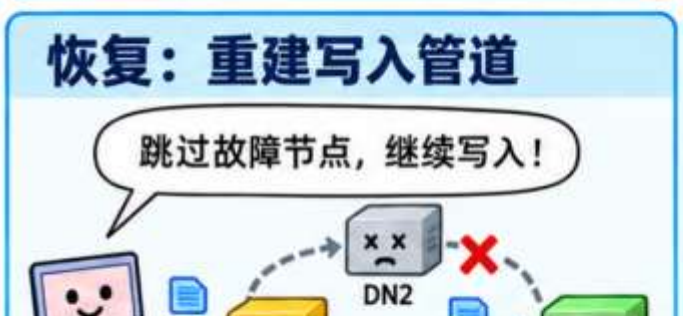

应答逐级返回客户端

## 异常：检测写入故障



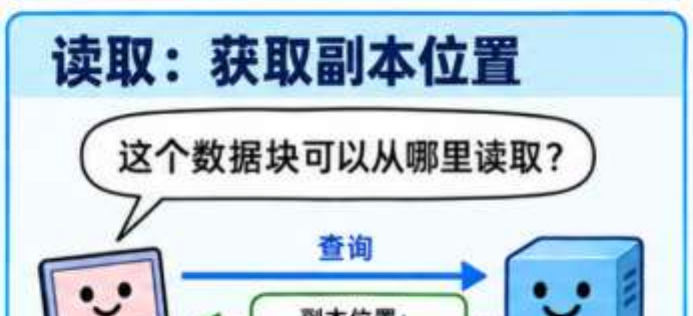

识别故障，准备重建管道

## 恢复：重建写入管道




未确认数据包重新发送
副本不足时，系统安排补齐

## 读取：获取副本位置




NameNode 返回该数据块的副本位置

## 切换：读取其他副本




节点故障时，尝试其他可用副本

# HDFS 的容错：故障类型和其检测方法。DATANODE 故障处理

首先，我必须向你介绍我储存的两张表。

| 数据块列表 |
| --- |
| 数据块 1- 储存在 DN1，DN2, DN3 |
| 数据块 2- 储存在 DN1，DN4, DN5 |

| DATANODE 列表 |
| --- |
| DATANODE1 - 存储数据块 1, 2 |
| DATANODE2 - 存储数据块 1, 5 |

我持续的更新这两张表 ...

如果我发现一个 DATANODE 上的数据块已经损坏，我更新第一张表（将这个 DN 从数据块列表中移除）

如果我发现某个 DATANODE 挂掉了，我会同时更新两张表

## 未充分备份的数据块

我定时扫描第一个列表（数据块列表），检查是否有数据块没有被正确地备份。

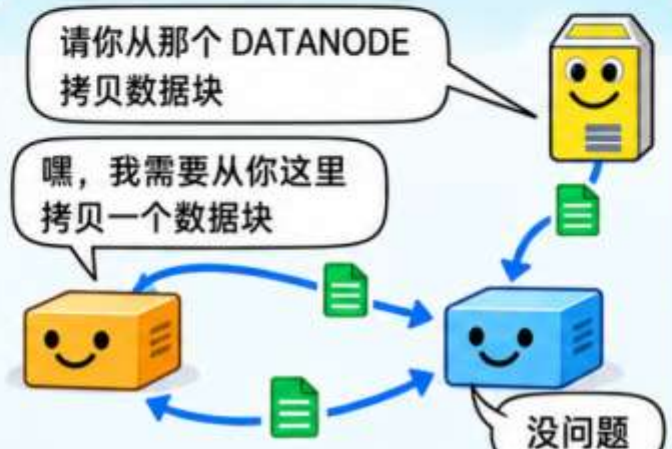
这些数据块被称作为充分备份的数据块。

对于所有未被充分备份的数据块，我指定另外的 DATANODE 从其他 DATANODE 中获得备份。

例如 ...

请你从那个 DATANODE 拷贝数据块

嘿，我需要从你这里拷贝一个数据块



没问题

嗯 .... 有一个问题：所有的这一切都是基于假设在某个地方已经存储了一份有效的数据块备份了 ... 对吗？



没错！ HDFS 不能保证至少有一份备份存活着，但是它会尽力智能地选择备份位置 ... 这一点我们接下来讲解 ...

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# 目 录




1 Hadoop简介

2 分布式文件系统HDFS

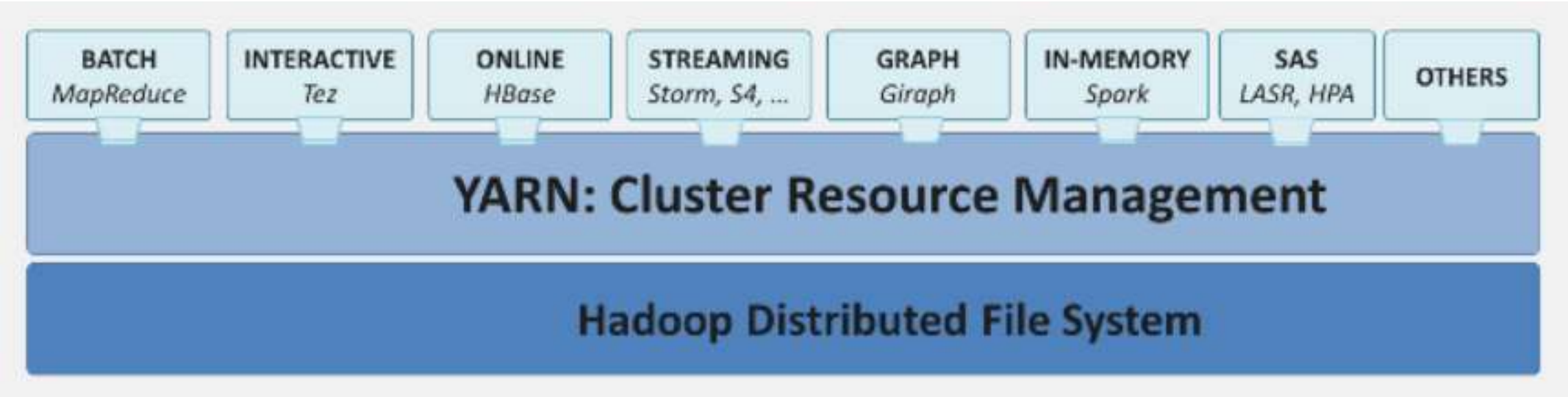
3 资源管理平台YARN

4 分布式计算框架MapReduce

5 搭建Hadoop

# 资源管理平台YARN

- **YARN——Yet Another Resource Negotiator，是Hadoop的资源管理层**
- **Yarn** 允许**不同的数据处理引擎**（*如图形处理、交互式处理、流处理以及批处理*）运行和处理**存储在HDFS中的数据**。除了资源管理之外，Yarn **还负责作业调度**



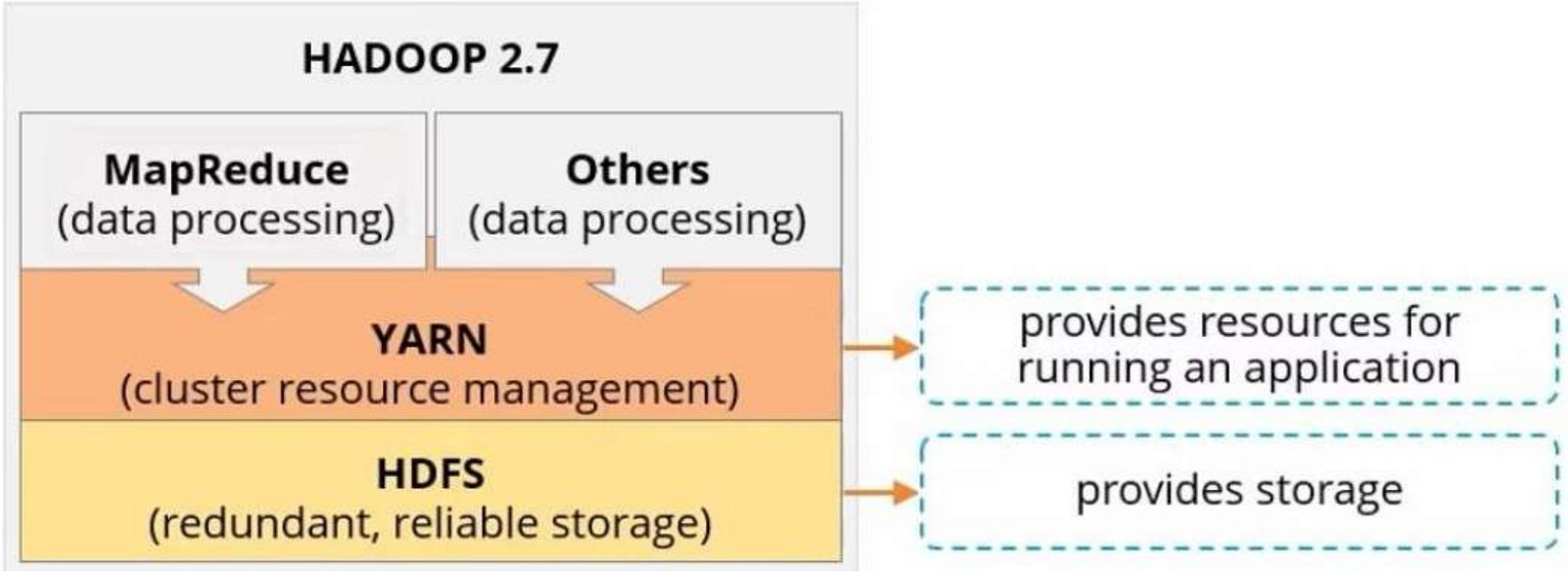
## YARN-Use Case

- 雅虎是第一家采用 Hadoop 的公司,并成为 Hadoop 生态系统中的潮流引领者。2012 年底，由于 MapReduce 的限制，雅虎在 Hadoop 架构上进行迭代和流处理遇到了困难。
  - 用户可以使用 Java、Python 和 Ruby 等脚本语言编写 MapReduce 程序，**无论使用什么语言，其实现都依赖于MapReduce处理模型**
  - 2012年5月发布的Hadoop 2.0版本引入了YARN，能够支持多种处理模型比如Spark
- 雅虎在 2013 年第一季度实施YARN以后安装了超过 30,000 个生产节点
  - Spark 用于迭代处理
  - Storm 用于流处理
  - Hadoop 用于批处理
- **集群的利用率更高，降低了运营成本，减少了数据移动**



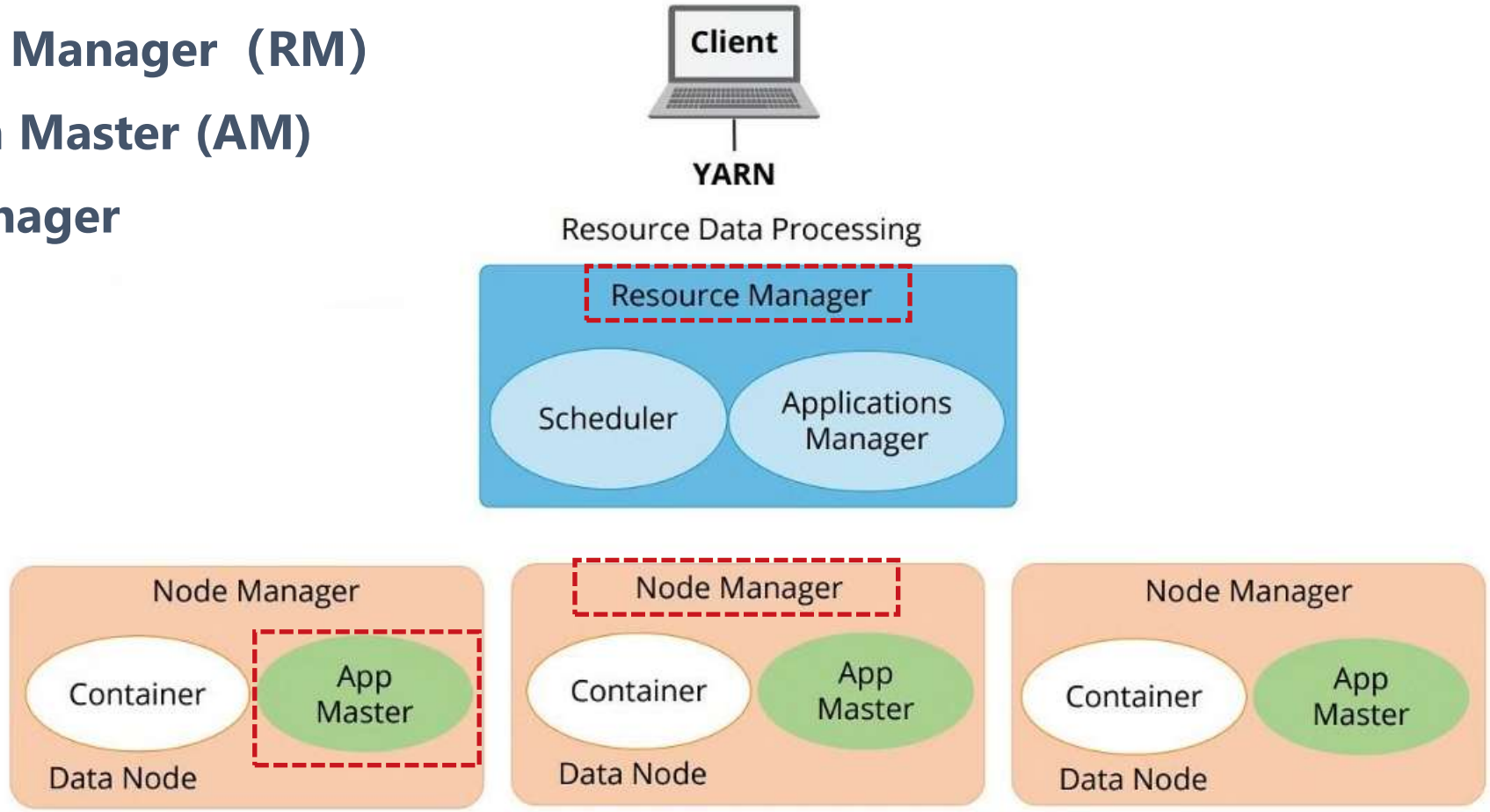
# 资源管理平台YARN

- YANR负责**提供应用程序执行所需的计算资源**，如GPU或者内存
  - YARN和HDFS分开，而**后者只提供存储**



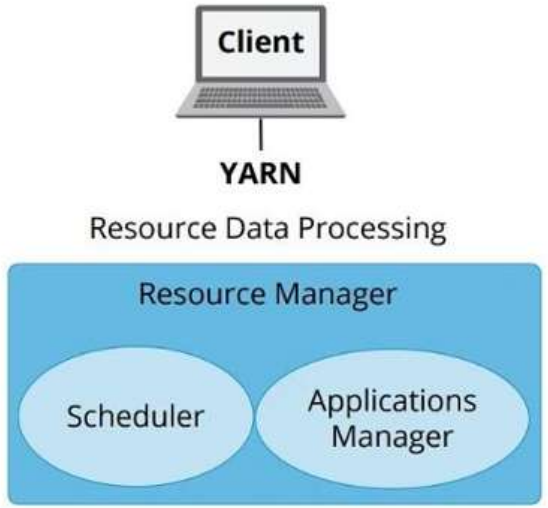
## YARN的架构

- 资源管理器 Resource Manager（RM）
- 应用大师 Application Master (AM)
- 节点管理器 NodeManager



## 资源管理器 Resource Manager（RM）

- 协调集群中的可用资源，以实现最大集群利用率
- **两个组件：资源调度程序(Scheduler)和应用程序管理器（Applications Manager)**
  - 调度程序负责根据容量、队列等常见约束将资源分配给各种正在运行的应用程序
  - 应用程序管理器是一个界面，用于维护已提交、当前正在运行或已完成的应用程序列表

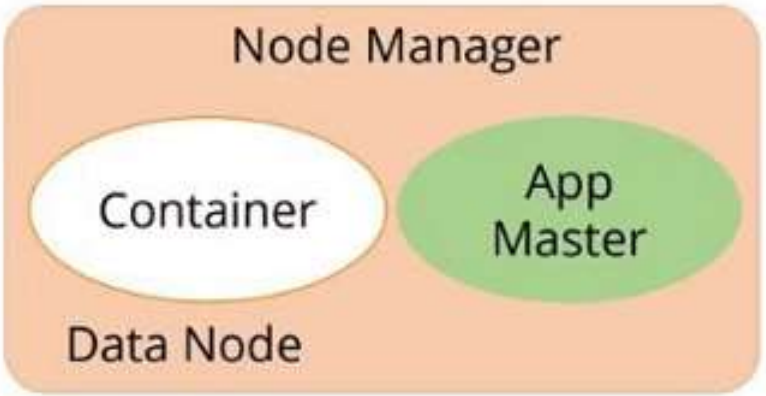


## 节点管理器Node Manager

- NodeManager 设置容器环境
- 监视节点的健康状况，当发生硬件或软件问题时向资源管理器报告，以便调度程序可以将资源分配转移到健康的节点，直到问题得到解决

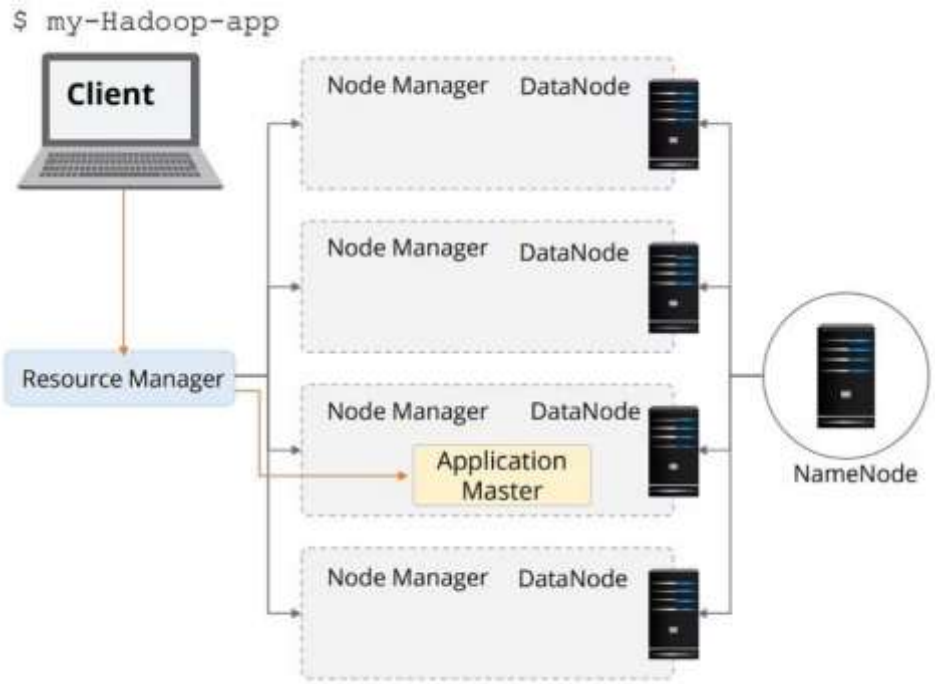
## 应用大师 Application Master (AM)

- 从 RM 协商资源并与 NodeManager 一起执行和监视容器及其资源消耗



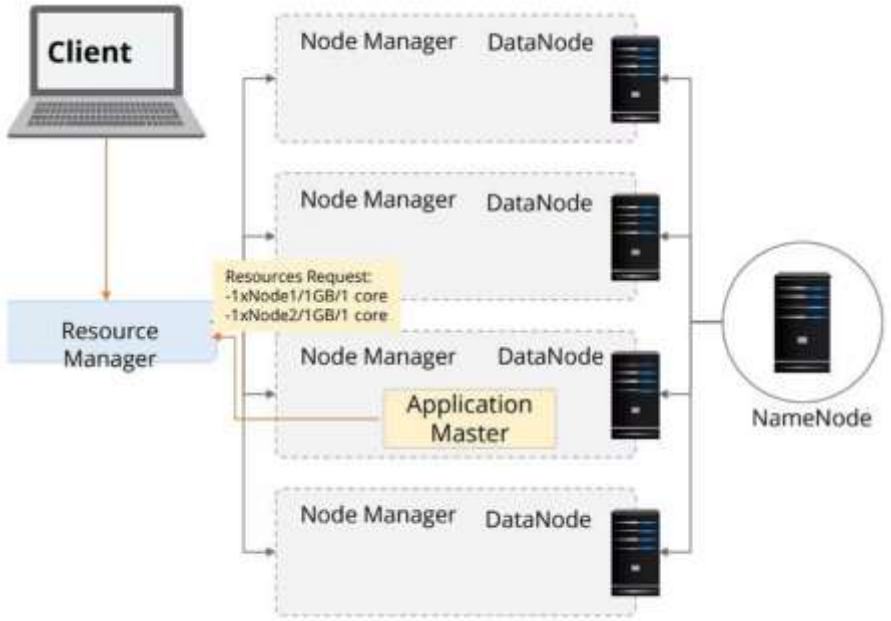
## 通过YARN运行应用程序的示例

- 第 1 步 - 向资源管理器Resource Manager（RM）提交申请
  - RM维护集群上的**应用程序列表**和**节点管理器上的可用资源**
  - RM确定**下一个**接收部分集群资源的**应用程序**。该决定**受到许多限制**，例如队列容量、访问控制列表和公平性。



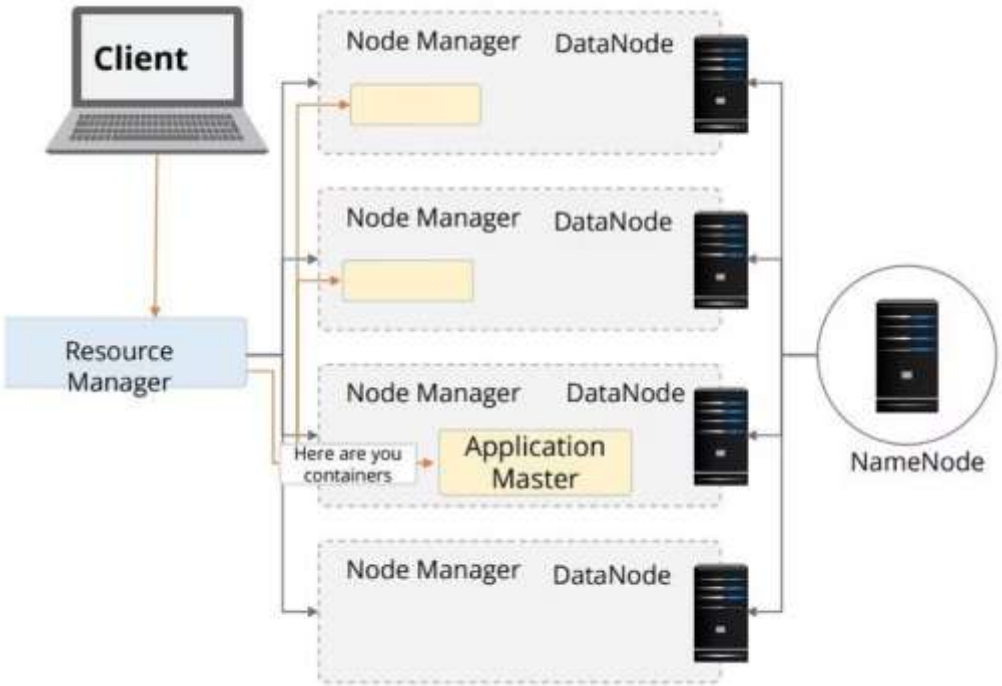
## 通过YARN运行应用程序的示例

- 第 2 步 - 资源管理器分配容器
  - 调度程序(scheduler)选择容器
  - Application Master向 RM发送资源请求，请求容器来运行应用程序的任务
  - RM**通过提供容器ID和主机名来分配容器**，满足Application Master的要求



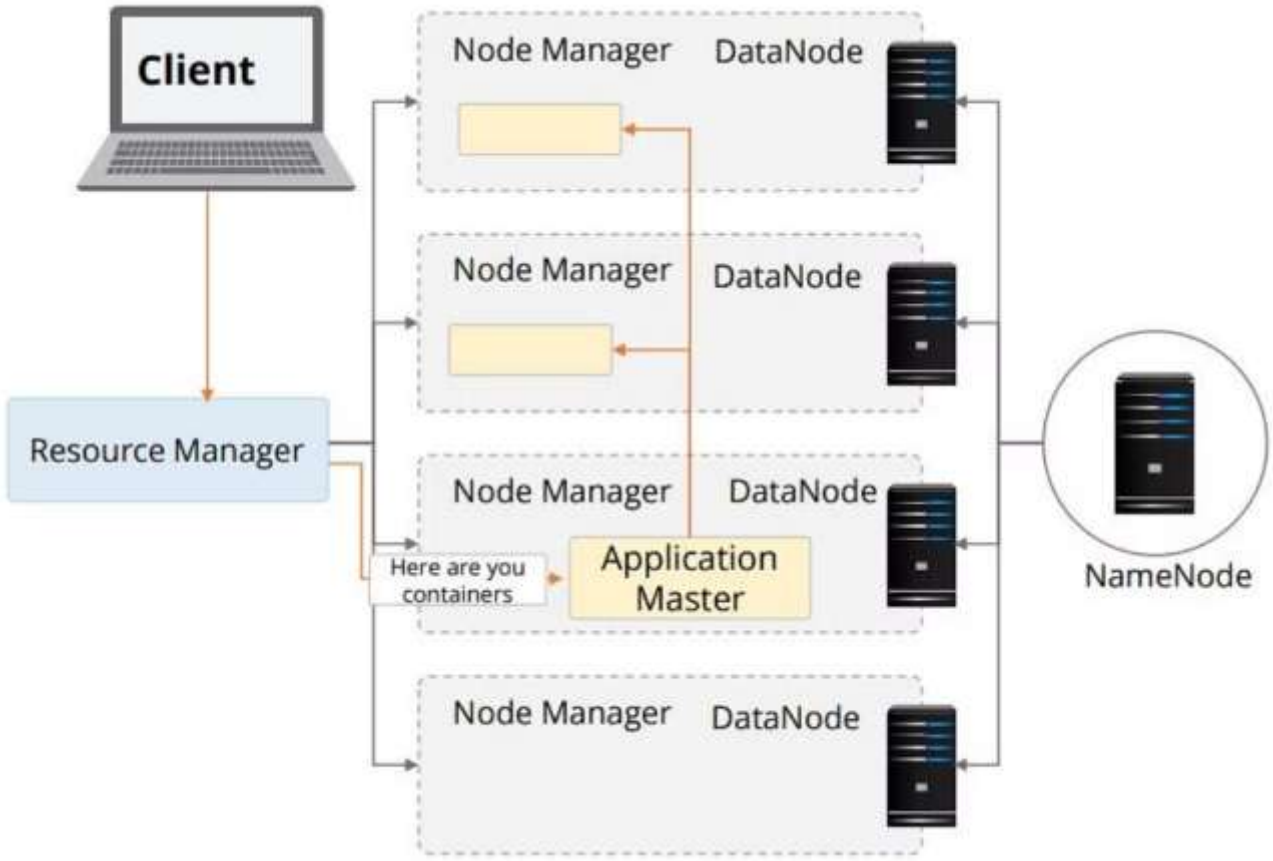
## 通过YARN运行应用程序的示例

- 第 3 步 - 应用程序主机联系节点管理器
  - 分配容器后，**应用程序主机**会**要求**管理分配容器的主机的**节点管理器**使用这些资源来启动特定于应用程序的任务。该任务可以是任何框架中编写的任何流程，例如MapReduce任务。



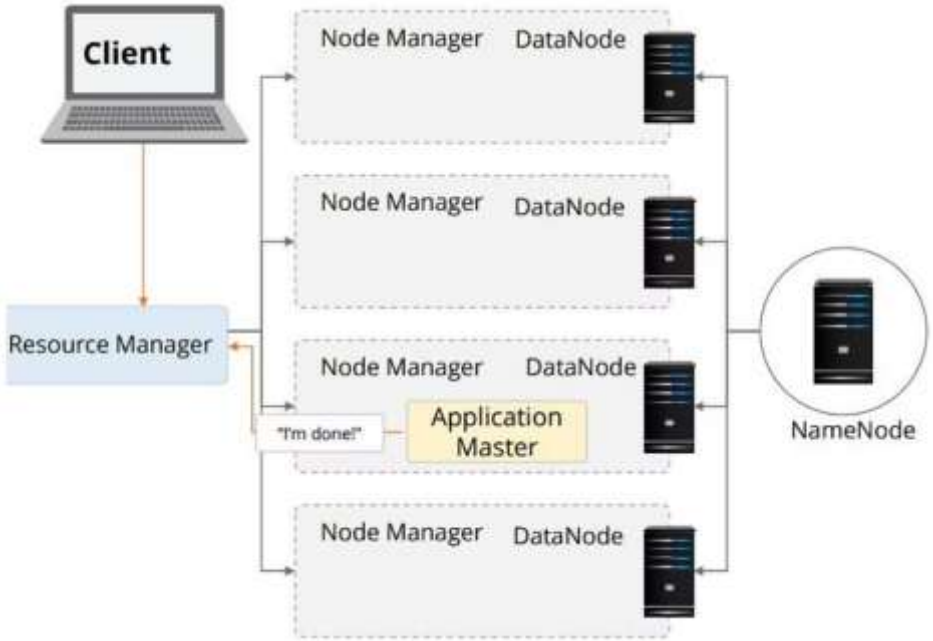
## 通过YARN运行应用程序的示例

- 第 4 步 - 资源管理器RM启动容器
  - NodeManager不监控任务；它仅监视容器中的资源使用情况



## 通过YARN运行应用程序的示例

- 第 5 步 - 容器执行应用程序主程序
  - 应用程序完成后，Application Master 会自行关闭并释放自己的容器
  - RM会检查Application Master 的运行状况，如果Application Master发生故障，它可以由RM在新容器中重新启动
  - RM负责管理 Application Master，而 Application Master 则负责管理任务



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# 目 录




1 Hadoop简介

2 分布式文件系统HDFS

3 资源管理平台YARN

4 分布式计算框架MapReduce

5 搭建Hadoop

## 洋葱辣椒酱与MapReduce

- 知乎作者分享的一个**关于一个程序员是如何给妻子讲解什么是MapReduce的例子**，可以更好地理解MapReduce

**我问妻子**：“你真的想要弄懂什么是MapReduce？”她很坚定的回答说“是的。”
**因此我问道**：你是如何准备洋葱辣椒酱的？
**妻子**： 我会取一个洋葱，把它切碎，然后拌入盐和水，最后放进混合研磨机里研磨。这样就能得到洋葱辣椒酱了。
**妻子**： 但这和MapReduce有什么关系？
**我**： 你等一下。让我来编一个完整的情节，这样你肯定可以在15分钟内弄懂MapReduce。



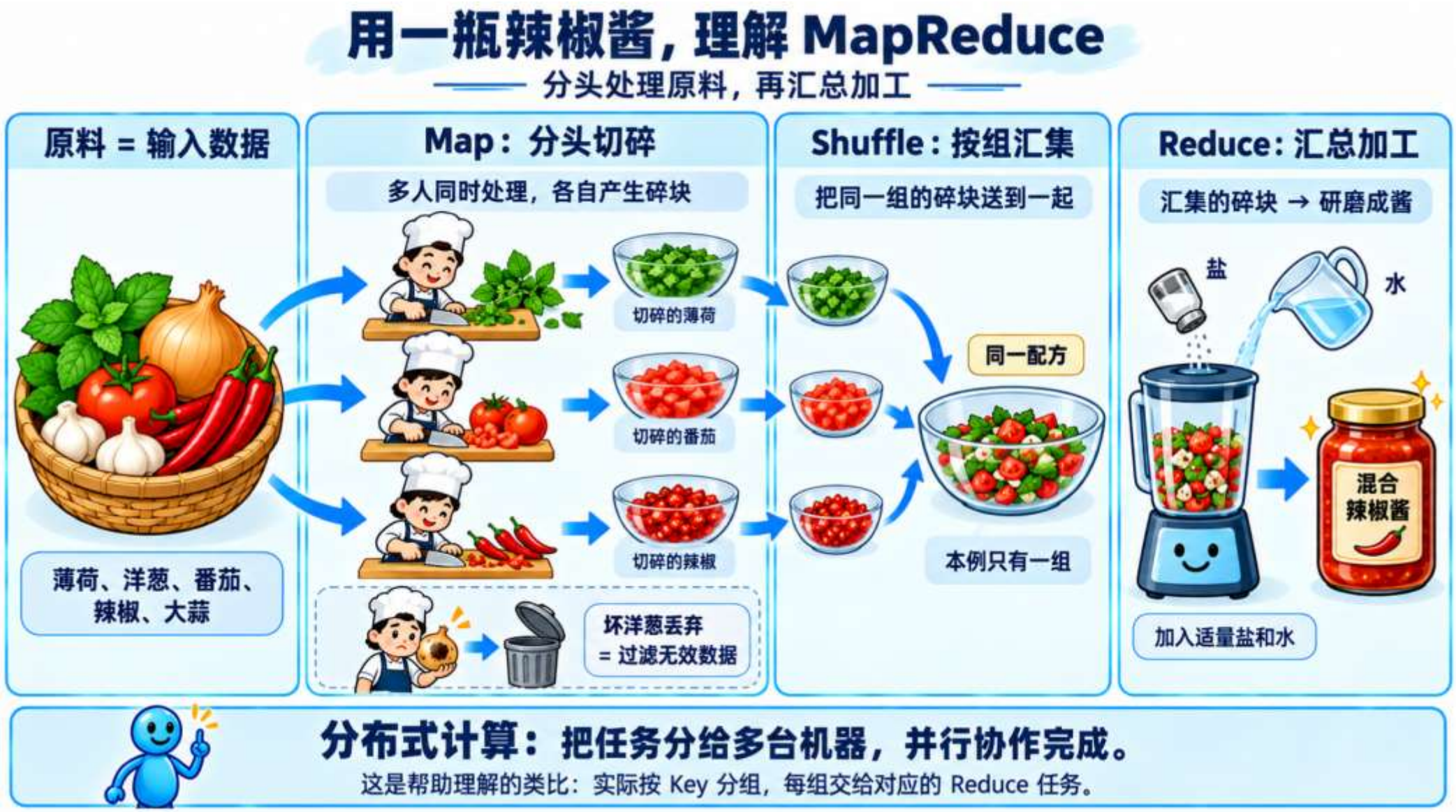
**我**：现在，假设你想用薄荷、洋葱、番茄、辣椒、大蒜弄一瓶混合辣椒酱。你会怎么做呢?
**妻子**： 我会取薄荷叶一撮，洋葱一个，番茄一个，辣椒一根，大蒜一根，切碎后加入适量的盐和水，再放入混合研磨机里研磨，这样你就可以得到一瓶混合辣椒酱了。
**我**： 没错，让我们把MapReduce的概念应用到食谱上。Map和Reduce其实是两种操作，我来给你详细讲解下。
*Map（映射）*：把洋葱、番茄、辣椒和大蒜切碎，是各自作用在这些物体上的一个Map操作。所以你给Map一个洋葱，Map就会把洋葱切碎。 同样的，你把辣椒，大蒜和番茄一一地拿给Map，你也会得到各种碎块。 所以，当你在切像洋葱这样的蔬菜时，你执行就是一个Map操作。 Map操作适用于每一种蔬菜，它会相应地生产出一种或多种碎块，在我们的例子中生产的是蔬菜块。在Map操作中可能会出现有个洋葱坏掉了的情况，你只要把坏洋葱丢了就行了。所以，如果出现坏洋葱了，Map操作就会过滤掉坏洋葱而不会生产出任何的坏洋葱块。
*Reduce（化简）*:在这一阶段，你将各种蔬菜碎都放入研磨机里进行研磨，你就可以得到一瓶辣椒酱了。这意味要制成一瓶辣椒酱，你得研磨所有的原料。因此，研磨机通常将map操作的蔬菜碎聚集在了一起。
**妻子**： 所以，这就是MapReduce?
**我**： 你可以说是，也可以说不是。 其实这只是MapReduce的一部分，MapReduce的强大在于分布式计算。





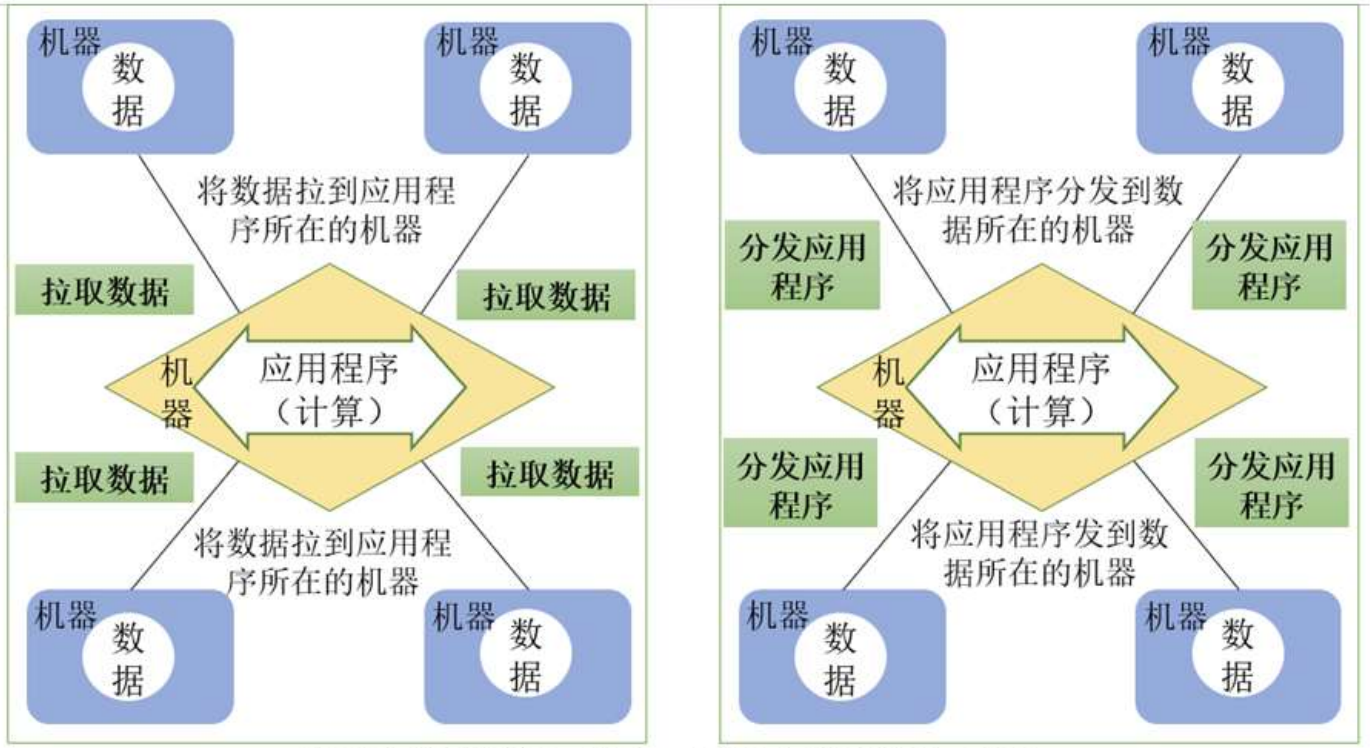
# 分布式计算框架MapReduce

**我**： 假设你参加了一个辣椒酱比赛，并且你的食谱赢得了最佳辣椒酱奖。得奖之后，辣椒酱食谱大受欢迎，于是你想要开始出售自制品牌的辣椒酱。假设你每天需要生产10000瓶辣椒酱，你会怎么办呢?
**妻子**： 我会找一个能为我大量提供原料的供应商。
**我**：是的，就是那样的。那你能否独自完成制作呢？也就是说，独自将原料都切碎？仅仅一部研磨机又是否能满足需要？而且现在，我们还需要供应不同种类的辣椒酱，像洋葱辣椒酱、青椒辣椒酱、番茄辣椒酱等等。
**妻子**： 当然不能了，我会雇佣更多的工人来切蔬菜。我还需要更多的研磨机，这样我就可以更快地生产辣椒酱了。
**我**：没错，所以现在你就不得不分配工作了，你将需要几个人一起切蔬菜。每个人都要处理满满一袋的蔬菜，而每一个人都相当于在执行一个简单的Map操作。每一个人都将不断地从袋子里拿出蔬菜来，并且每次只对一种蔬菜进行处理，也就是将它们切碎，直到袋子空了为止。
这样，当所有的工人都切完以后，工作台（每个人工作的地方）上就有了洋葱块、番茄块和蒜蓉等等。
**妻子**：但是我怎么会制造出不同种类的番茄酱呢?
**我**：现在你会看到MapReduce遗漏的阶段——搅拌阶段。MapReduce将所有输出的蔬菜碎都搅拌在了一起，这些蔬菜碎都是在以key为基础的map操作下产生的。搅拌将自动完成，你可以假设key是一种原料的名字，就像洋葱一样。所以全部的洋葱keys都会搅拌在一起，并转移到研磨洋葱的研磨器里。这样，你就能得到洋葱辣椒酱了。同样地，所有的番茄也会被转移到标记着番茄的研磨器里，并制造出番茄辣椒酱。


用一瓶辣椒酱，理解 MapReduce
分头处理原料，再汇总加工
原料 = 输入数据
薄荷、洋葱、番茄、辣椒、大蒜
Map：分头切碎
多人同时处理，各自产生碎块
切碎的薄荷
切碎的番茄
切碎的辣椒
坏洋葱丢弃
= 过滤无效数据
Shuffle：按组汇集
把同一组的碎块送到一起
同一配方
本例只有一组
Reduce：汇总加工
汇集的碎块 → 研磨成酱
盐
水
混合辣椒酱
加入适量盐和水
分布式计算：把任务分给多台机器，并行协作完成。
这是帮助理解的类比：实际按 Key 分组，每组交给对应的 Reduce 任务。

# 分布式计算框架MapReduce

- MapReduce将复杂的、运行于大规模集群上的并行计算过程高度抽象到了两个函数：**Map**和**Reduce**，这两个函数及其核心思想都源自函数式编程语言。
- MapReduce设计的一个理念就是“**计算向数据靠拢**”，而不是“数据向计算靠拢"，因为数据需要大量的网络传输开销，尤其是在大规模数据环境下，这种开销尤为惊人，所以，移动计算要比移动数据更加经济。在这种理念下，只要有可能，**一个集群中的MapReduce框架就会将Map程序就近地在HDFS数据所在的节点运行，即将计算节点和存储节点放在一起运行，从而减少了节点间的数据移动开销。**



数据向计算靠拢（左）、计算向数据靠拢（右）

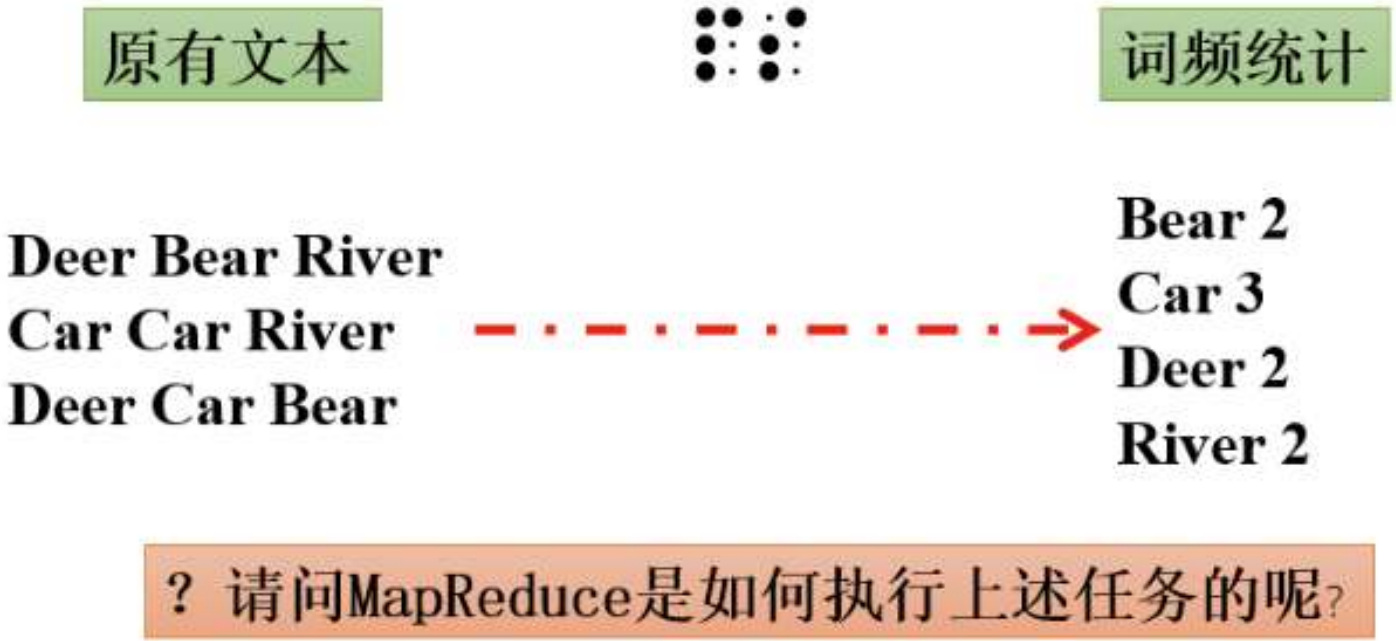
- 在整个MapReduce作业的过程中，包括数据的输入、**数据的处理**、数据的输出这些部分，而其中**数据的处理**部分就由map、reduce、combiner等操作组成。在一个MapReduce的作业中必定会涉及到如下一些组件：
  - 客户端：提交MapReduce作业
  - yarn资源管理器：负责集群上计算资源的协调
  - yarn节点管理器：负责启动和监控集群中机器上的计算容器（container）
  - MapReduce的application master：负责协调运行MapReduce的作业
  - HDFS：分布式文件系统，负责与其他实体共享作业文件

- MapReduce模型的核心是**Map函数和Reduce函数**。Map函数和Reduce函数都是以<key, value>作为输入，按一定的映射规则转换成另一个或一批<key, value>进行输出。

| 函数 | 输入 | 输出 | 说明 |
|---|---|---|---|
| Map | <$k_1$,v$_1$><br>如：<br><行号,"a b c"> | List(<$k_2$,$v_2$>)<br>如：<br><"a",1><br><"b",1><br><"c",1> | 1.将小数据集进一步解析成一批<key,value>对，输入Map函数中进行处理<br>2.每一个输入的<$k_1$,$v_1$>会输出一批<$k_2$,$v_2$>。<$k_2$,$v_2$>是计算的中间结果 |
| Reduce | <$k_2$,List($v_2$)><br>如：<br><"a",<1,1,1>> | <$k_3$,$v_3$><br><"a",3> | 输入的中间结果<$k_2$,List($v_2$)>中的List($v_2$)表示是一批属于同一个$k_2$的value |

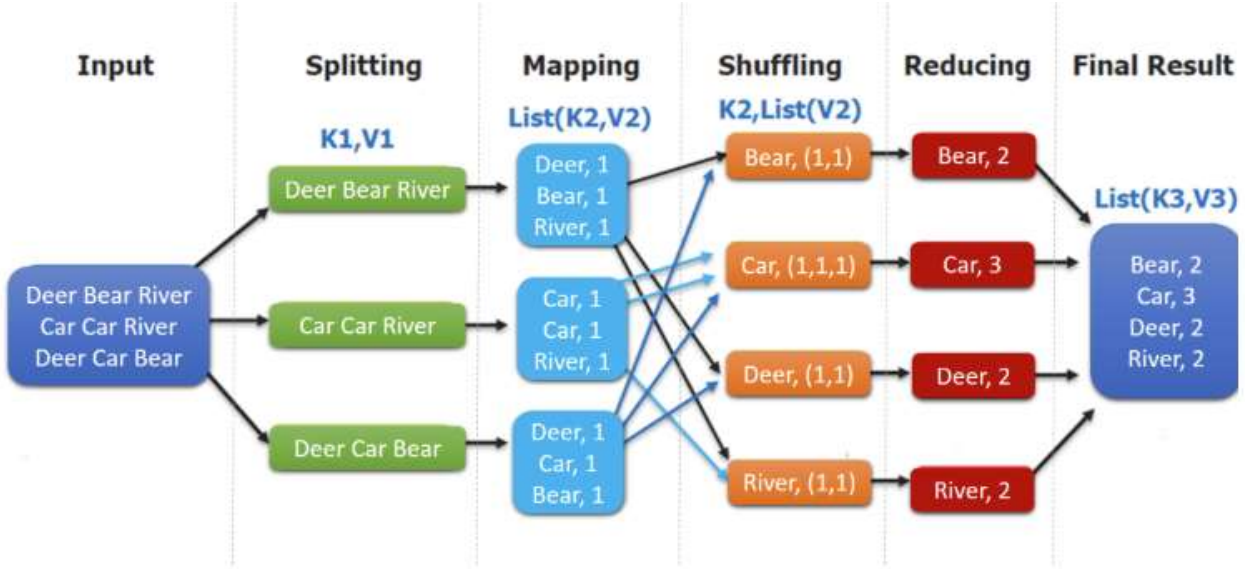
## 一个简单WordCount实例

- 我们想编写一个MapReduce程序，用于**统计一个文本文件中每个单词出现的次数**



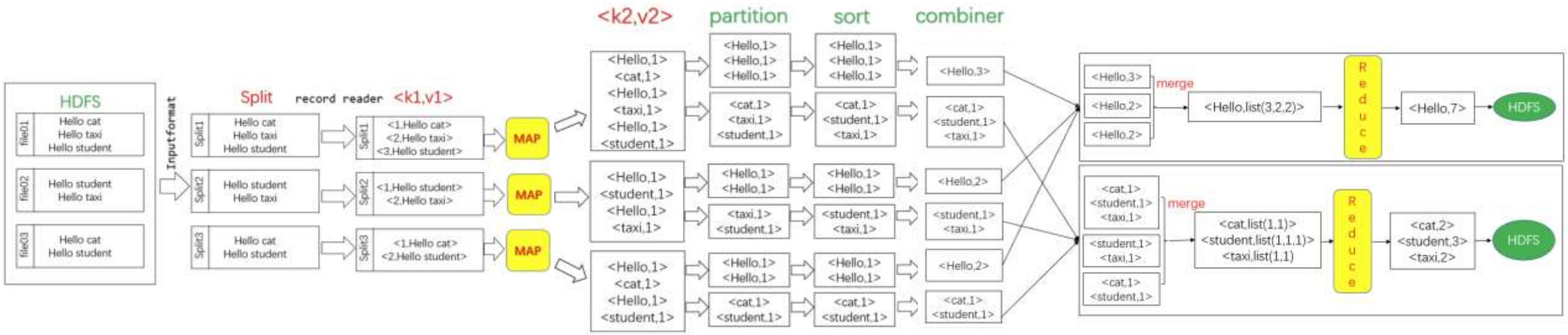
# 分布式计算框架MapReduce

- 对于Map函数的输入<k1,v1>而言，其具体**输入数据**就是<某一行文本在文件中的偏移位置，该行文本的内容>。用户可以自己编写Map函数处理过程，把文件中的一行读取后解析出每个单词，**输出**一批中间结果<单词，出现次数>
- 然后，这些中间结果会被分发到多个Reduce任务在多台机器上**并行执行**，**具有相同key**的<key,value>会被发送到同一个Reduce任务那里，Reduce任务会对中间结果进行汇总计算得到最后结果，并输出到分布式文件系统中。




Shuffle**是指针对Map输出结果进行分区、排序和合并等处理**，并交给Reduce的过程。因此，Shuffle过程分为Map端的操作和Reduce端的操作。

## 分布式系统下的MapReduce计算过程



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# 目 录




1 Hadoop简介

2 分布式文件系统HDFS

3 资源管理平台YARN

4 分布式计算框架MapReduce

5 搭建Hadoop

# Hadoop环境搭建

## Hadoop的安装模式：单机、伪分布式、完全分布式、HA等

## 1. 软件准备

- 安装Java (jdk8)
- 安装SSH

## 2. 环境准备

- Master和slave节点
- 网络连接方式和IP地址设置
- 注意防火墙
- SSH免密登录

## 3. 安装Hadoop （单机模式）

- 下载Hadoop (3.1.3)文件
- 解压缩
- 配置Hadoop运行账户（root）
- 配置Hadoop环境变量
- 执行source命令使配置生效

## 4. 伪分布式模式

- 配置HDFS集群
  - Core-site.xml文件和hdfs-site.xml文件
- 格式换文件系统
- 启动HDFS，查看进程以及HDFS Web页面

## 伪分布式模式下的YARN集群

- Mapred-site.xml文件和yarn-site.xml文件
- 用浏览器查看YARN Web页面
- 使用YARN调度MapReduce作业

## 在容器中部署Hadoop集群

- 创建1个容器作为master，部署好Hadoop并打包成镜像
- 在master中运行NameNode, SecondaryNameNode, ResourceManager三个进程
- 基于此镜像再创建2个容器，运行DataNode、NodeManager进程，将其作为slave nodes

## 一些参考链接

- https://cloud.tencent.com/developer/article/1998425
- https://blog.csdn.net/CSDN1csdn1/article/details/129853759
- https://www.cnblogs.com/fangts/p/16880953.html

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

# Thank You

